# Subject : LP-3

- Section : Machine Learning
- Experiment : ML - 4

```
import pandas as pd
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score,
    precision_score, recall_score
)
```

```
uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

print("Dataset Shape:", df.shape)
print(df.head())
```

```
Choose Files diabetes.csv
diabetes.csv(text/csv) - 23105 bytes, last modified: 21/10/2026 - 100% done
Saving diabetes.csv to diabetes.csv
Dataset Shape: (768, 9)
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

```
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

print("Features Shape:", X.shape)
print("Target Shape:", y.shape)
print("\nClass Distribution:")
print(y.value_counts())
```

```
Features Shape: (768, 8)
Target Shape: (768,)

Class Distribution:
Outcome
0    500
```

```
1    268
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (614, 8)
Testing Data: (154, 8)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Data normalization completed.")
```

```
Data normalization completed.
```

```
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("KNN model trained successfully.")
```

```
KNN model trained successfully.
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("Accuracy:", round(accuracy, 4))
print("Error Rate:", round(1 - accuracy, 4))

print("Precision:",
      round(precision_score(y_test, y_pred, zero_division=0), 4))

print("Recall:",
      round(recall_score(y_test, y_pred, zero_division=0), 4))
```

```
Confusion Matrix:
[[80 20]
 [26 28]]
Accuracy: 0.7013
Error Rate: 0.2987
Precision: 0.5833
Recall: 0.5185
```